

Clase 232 — Seguridad serverless

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: *OWASP Serverless Top 10 y documentación de AWS Lambda / Azure Functions / Google Cloud Functions* 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

Objetivo

Asegurar cargas serverless (funciones como servicio) entendiendo cómo cambia el modelo de amenazas cuando no hay servidor que endurecer: la superficie se traslada al código de la función, a sus permisos IAM, a sus disparadores (triggers) y a sus dependencias. El alumno aplicará privilegio mínimo por función, gestión segura de secretos y controles frente a inyección y abuso.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Describir** el modelo de amenazas serverless y el OWASP Serverless Top 10.
2. **Aplicar** privilegio mínimo por función (un rol IAM por función).
3. **Proteger** disparadores (API Gateway, colas, buckets) frente a abuso.
4. **Gestionar** secretos y variables de entorno de forma segura.
5. **Mitigar** inyección, dependencias vulnerables y denial-of-wallet.

Temas

#	Tema	Por qué importa
1	Modelo de amenazas serverless	La superficie se mueve al código y a IAM
2	Permisos por función	Un rol excesivo = escalada si la función cae
3	Event injection	Los eventos vienen de fuentes no confiables
4	Secretos en funciones	Variables de entorno no son un vault
5	Dependencias y cadena de suministro	Paquetes vulnerables en el bundle
6	Denial-of-Wallet	Abuso que dispara el coste, no la caída
7	Observabilidad de funciones	Trazas y logs para detección

Definiciones y características

- **Función como servicio (FaaS):** código que corre bajo demanda sin gestionar servidores. *Clave:* no parcheas SO, pero sí el código y sus permisos.
- **Trigger/evento:** fuente que invoca la función (HTTP, cola, objeto en bucket). *Clave:* el payload del evento es entrada no confiable.

- **Rol de ejecución:** identidad IAM que asume la función. *Clave:* debe ser mínimo y exclusivo por función.
- **Event injection:** inyección a través del payload del evento. *Clave:* valida y sanea todo evento entrante.
- **Denial-of-Wallet:** abuso que multiplica invocaciones y coste. *Clave:* limita concurrencia y presupuesto.
- **Cold start:** primera invocación tras inactividad. *Clave:* relevante para timeouts y para no cachear secretos inseguros.
- **Variables de entorno:** config inyectada en la función. *Clave:* no son secretas por sí solas; usa un gestor de secretos y cifrado.

Herramientas y preparación

- Una cuenta de laboratorio con AWS Lambda (o Azure Functions / Cloud Functions).
- **Serverless Framework** o **SAM** para desplegar; **cfn-nag/Checkov** para revisar plantillas.
- Un escáner de dependencias (p. ej. `npm audit`, `pip-audit`, Trivy) y un gestor de secretos (clase 233).

```
# Revisar los permisos del rol de ejecución de una Lambda
aws lambda get-function --function-name mi-func \
  --query 'Configuration.Role'
# Escanear dependencias del paquete de la función
pip-audit -r requirements.txt
```

Laboratorio guiado

1. Despliega una función simple (p. ej. procesa un objeto subido a un bucket) con Serverless Framework en tu cuenta de laboratorio.
2. Revisa su **rol de ejecución**: si tiene permisos amplios, recórtalo a solo las acciones necesarias (leer ese bucket, escribir en esa cola).
3. **Event injection:** añade a la función una consulta construida con datos del evento sin sanear y demuestra la inyección (en laboratorio); luego parametriza/valida la entrada y confirma la mitigación.
4. Mueve un "secreto" de una variable de entorno a un gestor de secretos (Secrets Manager/Key Vault) y recupéralo en runtime con permisos mínimos.
5. Escanea las dependencias del paquete con `pip-audit` / `npm audit` / Trivy y actualiza las vulnerables.
6. Configura **límites de concurrencia** y presupuesto/alarma de coste para mitigar denial-of-wallet.
7. Habilita trazas (X-Ray/Application Insights) y logs estructurados; provoca un error y compruébalo en el log.

Ejercicios

1. Reescribe un rol de ejecución amplio a privilegio mínimo para un caso concreto.
2. Valida y sanea el payload de un evento HTTP antes de usarlo en una consulta.
3. Migra tres secretos de variables de entorno a un gestor de secretos.
4. Añade un límite de concurrencia y una alarma de coste a una función.
5. Escanea el bundle de la función y corrige una dependencia vulnerable.
6. Diseña una regla de detección para invocaciones anómalas de una función.

Reto verificable

Endurece una función serverless de laboratorio: rol de ejecución mínimo y exclusivo, entrada validada, secretos fuera de las variables de entorno, dependencias sin CVEs críticas y límite de concurrencia.

Criterio de aceptación: el rol de la función solo permite las acciones estrictamente necesarias (verificable con la política), el ataque de event injection ya no funciona, no hay secretos en las variables de entorno y `pip-audit / npm audit` no reporta vulnerabilidades críticas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Función con <code>AdministratorAccess</code>	Rol reutilizado y excesivo; crea un rol mínimo por función.
Secreto legible en la consola	Guardado en variable de entorno en claro; muévelo a un gestor de secretos.
Factura disparada de repente	Denial-of-wallet o bucle de eventos; limita concurrencia y añade alarmas de coste.
Inyección vía payload del evento	Entrada no saneada; valida esquema y parametriza consultas.
CVE en una dependencia del bundle	Paquete vulnerable empaquetado; escanea y actualiza en cada build.

Preguntas frecuentes

? ¿Serverless es más seguro porque no hay servidor? El proveedor asume el parcheo del SO y del runtime, lo que elimina una clase de problemas. Pero la superficie se traslada al código, a los permisos IAM, a los eventos y a las dependencias, que siguen siendo tu responsabilidad.

? ¿Las variables de entorno sirven para secretos? No como almacén seguro: son visibles para quien pueda leer la configuración de la función y pueden filtrarse en logs. Usa un gestor de secretos y concede a la función permiso mínimo para leerlos en runtime.

? ¿Qué es denial-of-wallet y por qué preocupa en serverless? Es un abuso que no busca tumbar el servicio sino disparar el número de invocaciones y, con ello, el coste. Se mitiga con límites de concurrencia, throttling en el API Gateway y alarmas/presupuestos de coste.

Referencias

- OWASP Serverless Top 10. <https://owasp.org/www-project-serverless-top-10/>
- AWS Lambda — Security overview. <https://docs.aws.amazon.com/lambda/latest/dg/lambda-security.html>
- Azure Functions security. <https://learn.microsoft.com/azure/azure-functions/security-concepts>
- Google Cloud Functions — Securing. <https://cloud.google.com/functions/docs/securing>
- OWASP — Serverless Security Cheat Sheet. <https://cheatsheetseries.owasp.org/>

Siguiente clase

Clase 233 - Gestion de secretos en la nube